>_ Projeto final · LPOO · Licenciatura em Computação · 2026.2

Projeto IPE

Plataforma de Iniciação, Pesquisa e Extensão

O Projeto IPE é a entrega final de Linguagem de Programação Orientada a Objetos: um sistema de console em Java, construído em **quatro entregas** entre 16/10 e 18/12. Cada entrega cobra só o que já foi visto em aula e **soma ao código da anterior**.

3 a 4
pessoas por equipe, formadas em 16/10

4 entregas
com nota, de 06/11 a 18/12

Java
programa de console, começa pelo `Main.java`

Defesa
individual, junto da prova P² em 18/12

Como o sistema funciona

1. **O professor publica** uma oportunidade de monitoria ou de pesquisa, com vagas
2. **O estudante se inscreve** nas oportunidades que ainda estão abertas
3. **O professor classifica** os inscritos até preencher as vagas

Cada entrega soma ao código da anterior

E1 · 06/11

Classes do domínio

- encapsulamento
- herança
- toString e @Override

classes novas

`Usuario`
`Professor`
`Estudante`
`Oportunidade`
`Inscricao`

E2 · 27/11

Tipos de oportunidade

- classe abstrata
- polimorfismo
- interface
- sobrecarga

classes novas

`Monitoria`
`Pesquisa`
`Resumivel`

E3 · 11/12

Regras e listas

- ArrayList
- String
- exceções

classe nova

`VagasEsgotadas-Exception`

Final · 18/12

Sistema e defesa

- HashMap no login
- menu com Scanner
- instanceof
- defesa oral

o que muda

`Main com menu sistema completo`

01 Regras gerais

O projeto é feito em equipe, em Java, e cresce a cada entrega sobre o mesmo código.

Equipes
De 3 a 4 pessoas, formadas em 16/10. Todos precisam conhecer o código inteiro: a defesa final é individual.

Linguagem e ferramenta
Java, programa de console, no OnlineGDB ou na IDE da equipe. O programa sempre começa pelo `Main.java`.

Nomes
Classes, atributos e métodos com os nomes deste documento, sem acento. Nomes extras são livres.

Como entregar
Um `.zip` com todos os `.java`, enviado no ambiente virtual da turma até as 23h59 da quinta-feira anterior à aula da entrega.

Conferência na aula
A equipe roda o `Main.java` na frente do professor, em até 5 minutos. O que não roda não pontua naquela entrega.

Atraso
Até uma semana vale metade; depois disso, zero.

Main de demonstração
Cada entrega diz o que o `Main.java` precisa mostrar. Ele é o roteiro da conferência: se está no roteiro, precisa aparecer na tela.

Toda entrega passa pela conferência na aula

1 · A equipe envia
um .zip com os .java
até quinta, 23h59

→

2 · Conferência na aula
a equipe roda o Main.java
até 5 minutos por equipe

→

3 · Nota
cada exigência conferida
uma por uma

↓

Atrasou?
até 1 semana: metade

Não rodou?
o que não roda não pontua

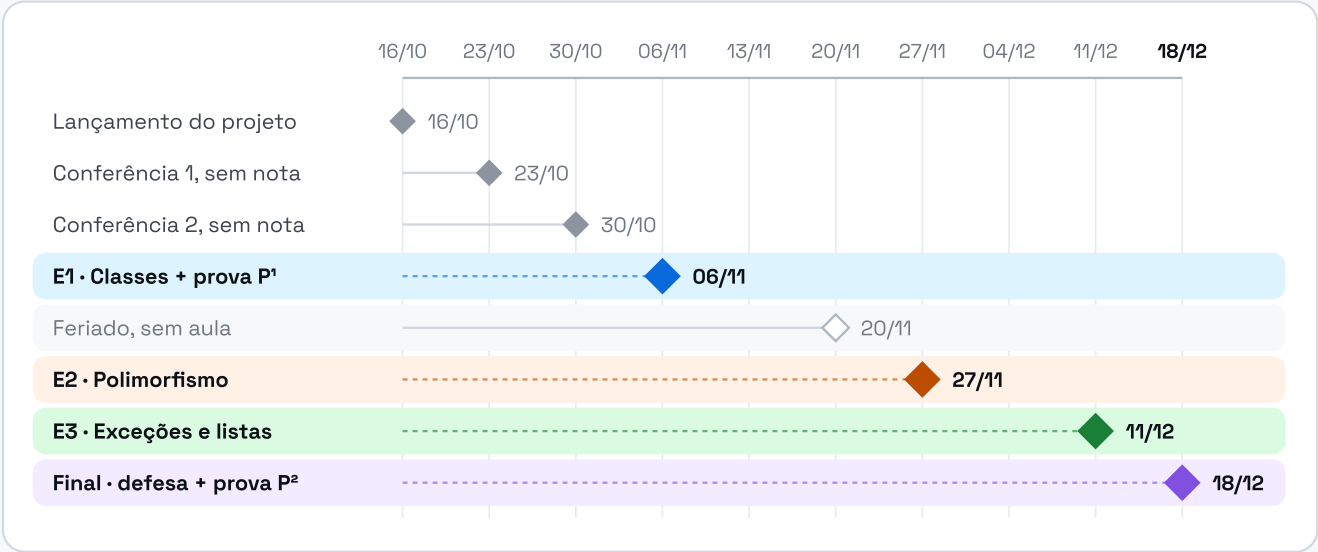
Mais de 1 semana?
a entrega vale zero

Em toda entrega, a correção confere

- [ ] O código compila e o `Main.java` roda do começo ao fim
- [ ] Cada item das exigências da entrega, um por um
- [ ] As regras do sistema garantidas dentro das classes: se o `Main` consegue furar uma regra, o item não conta
- [ ] Os nomes de classes e métodos iguais aos deste documento

02 Calendário

São quatro entregas com nota. As duas conferências de outubro não valem nota: servem para a equipe mostrar o andamento e corrigir o rumo antes da E1.



Em 20/11 não há aula. A E1 é conferida no mesmo dia da prova P¹, e a defesa acontece junto da prova P².

03 Avaliação

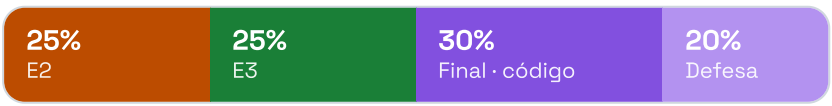
A E1 entra na N¹ como nota de listas de exercícios. As demais formam a nota do projeto (PD) na N².

E1 · 06/11

N¹

vale **100% da nota de listas** de exercícios

Nota do projeto (PD), na N² — 100%

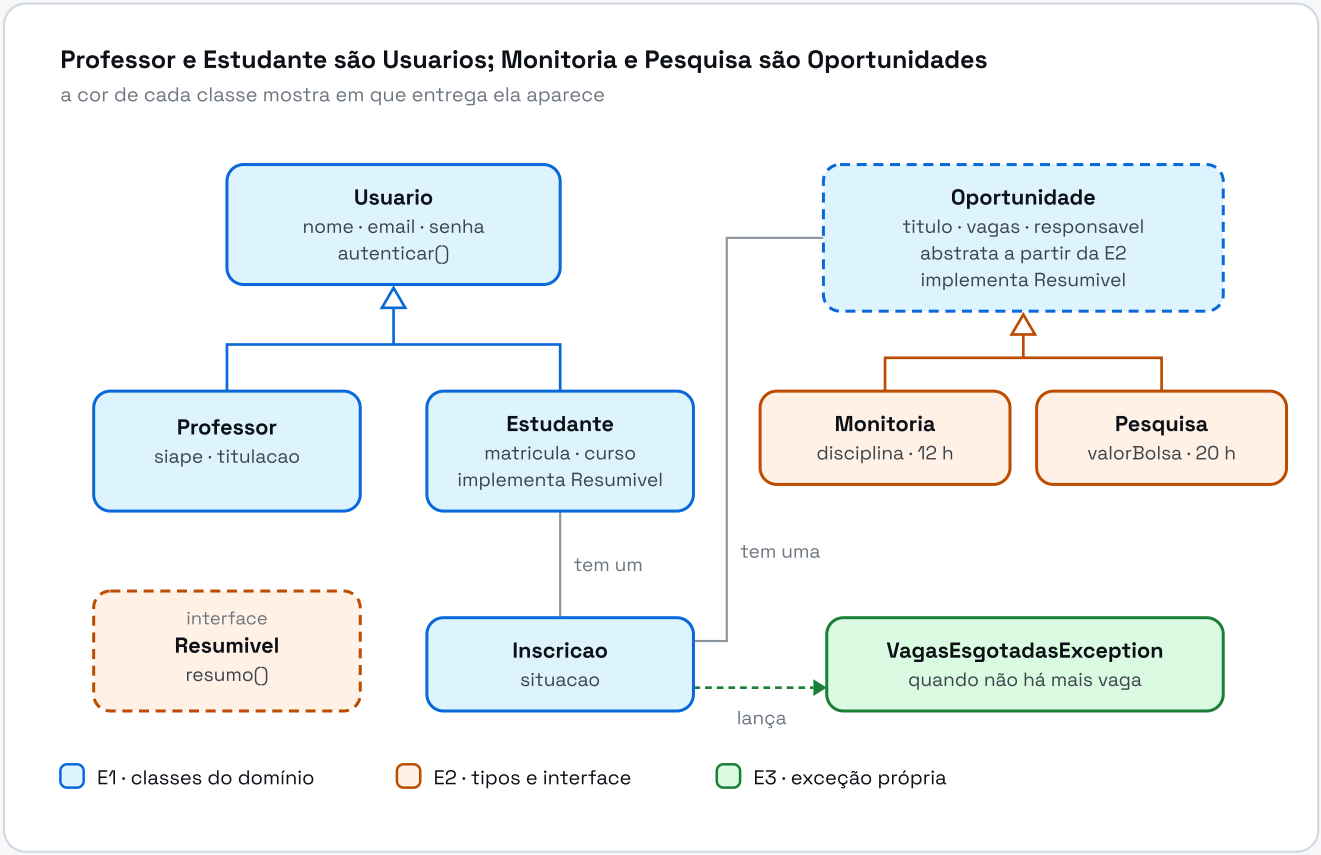


A defesa é individual: cada integrante tem a própria nota nesses 20%.

Entrega	Data	Compõe	Peso
E1 · Classes do domínio	06/11	N¹, listas de exercícios	100% das listas
E2 · Tipos de oportunidade	27/11	N², projeto	25% do PD
E3 · Regras e listas	11/12	N², projeto	25% do PD
Final · código	18/12	N², projeto	30% do PD
Final · defesa, individual	18/12	N², projeto	20% do PD

04 O sistema

Professores publicam oportunidades, estudantes se inscrevem e o professor classifica os inscritos até preencher as vagas. A versão final tem **sete classes, uma interface e uma exceção**: a E1 traz cinco classes, a E2 os dois tipos de oportunidade e a interface, a E3 a exceção.



Triângulo: herda de (é um). Linha simples: tem um. Tracejado: classe abstrata ou interface.

As regras que o código precisa garantir

R1 Uma oportunidade tem **pelo menos 1 vaga**.

R2 Só há inscrição em oportunidade **aberta**.

R3 Os classificados **nunca passam** do número de vagas.

R4 A senha nunca sai do objeto: **não existe** `getSenha`. A partir da E3, todo e-mail tem @.

E1 Classes do domínio

encapsulamento · herança · toString · @Override

06/11
sexta · com a P1

Cinco classes, com as regras guardadas dentro delas.

Classe	Atributos	Métodos exigidos
`Usuario`	nome, email, senha	`getNome`, `getEmail`, `autenticar(email, senha)`. Sem `getSenha`
`Professor` extends Usuario	siape, titulacao	getters
`Estudante` extends Usuario	matricula, curso	getters
`Oportunidade`	titulo, vagas, classificados = 0, aberta = true, responsavel	getters, `encerrar()`, `inscrever(estudante)`, `registrarClassificacao()`
`Inscricao`	estudante, oportunidade, situacao = "pendente"	getters, `classificar()`, `desclassificar()`

> `inscrever` devolve a nova `Inscricao`, ou `null` se a oportunidade estiver encerrada. `registrarClassificacao` e `classificar` devolvem `false` quando não há vaga. Se o construtor de `Oportunidade` receber menos de 1 vaga, a oportunidade fica com 1.

Exigências

- [ ] Todos os atributos `private`
- [ ] `Professor` e `Estudante` chamam `super(...)` na primeira linha do construtor
- [ ] Nenhum setter: tudo muda por métodos que guardam as regras
- [ ] Regras R1 a R4 garantidas dentro das classes, nunca no `Main`
- [ ] `toString` com `@Override` nas cinco classes

Main.java · o que precisa aparecer na tela

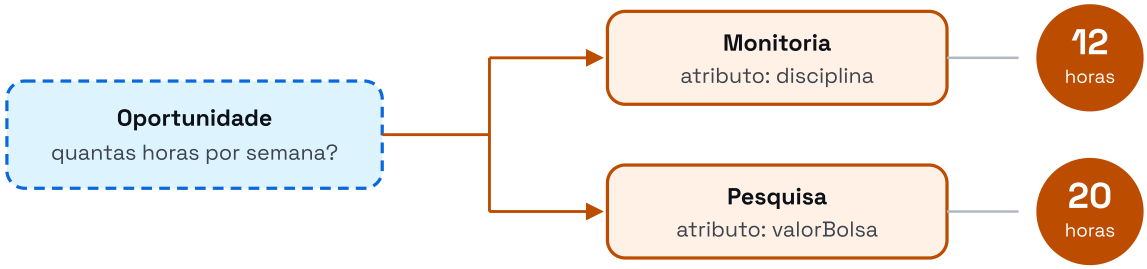
01 Um professor e dois estudantes, impressos com `toString`
02 `autenticar` com a senha certa e com a senha errada
03 Uma oportunidade de 1 vaga com duas inscrições: a primeira classificada e a segunda recusada
04 Uma inscrição depois de `encerrar()`, recusada

E2 Tipos de oportunidade

classe abstrata · polimorfismo · interface · sobrecarga

27/11 sexta

`Oportunidade` vira classe abstrata, e cada tipo responde do seu jeito à mesma pergunta.



A mesma pergunta, `cargaHorariaSemanal()`, feita a qualquer `Oportunidade`: cada tipo dá a sua resposta.

Subclasse de `Oportunidade`	Atributo próprio	`cargaHorariaSemanal()`
`Monitoria`	disciplina	12
`Pesquisa`	valorBolsa	20

Exigências

- [ ] `Oportunidade` é `abstract`, com o método abstrato `cargaHorariaSemanal()`
- [ ] `Monitoria` e `Pesquisa`, cada uma com seu atributo e chamando `super(...)`
- [ ] `toString` sobrescrito nas duas, aproveitando `super.toString()`
- [ ] Interface `Resumivel` com `String resumo()`, implementada por `Estudante` e por `Oportunidade`
- [ ] Um construtor sobrecarregado: `Monitoria` com e sem o número de vagas (sem o número, 1 vaga)

`Main.java · o que precisa aparecer na tela`

01 Um vetor `Oportunidade[]` com uma monitoria e uma pesquisa, percorrido num laço que imprime `toString` e `cargaHorariaSemanal()`

02 Um vetor `Resumivel[]` com um estudante e as duas oportunidades, impresso num laço com `resumo()`

E3 Regras e listas

ArrayList · String · exceções

11/12
sexta

As inscrições passam a ficar guardadas, e a falta de vaga deixa de devolver `false` para lançar uma exceção.

antes · E1 e E2

sem vaga → **`false`**

O `Main` precisa lembrar de olhar o resultado. Se esquecer, o erro passa despercebido.

a partir da E3

sem vaga → **`VagasEsgotadasException`**

O Java obriga quem chama a tratar o erro. Ninguém esquece.

Exigências

- [ ] `Oportunidade` guarda as inscrições num `ArrayList<Inscricao>` e tem `listarInscricoes()`, que imprime todas
- [ ] Exceção própria `VagasEsgotadasException`, que estende `Exception`, lançada por `registrarClassificacao()` e `classificar()` quando não há vaga; os dois métodos declaram `throws`
- [ ] `IllegalArgumentException`, do Java, para vagas menor que 1 e para e-mail sem @ (use `contains`)
- [ ] `autenticar` compara o e-mail sem diferença entre maiúsculas e minúsculas (use `equalsIgnoreCase`)
- [ ] Tudo das entregas anteriores continua funcionando

Main.java · o que precisa aparecer na tela

01 Três inscrições na mesma oportunidade, listadas com `listarInscricoes()`
02 A `VagasEsgotadasException` provocada e tratada com `try/catch`
03 A `IllegalArgumentException` provocada duas vezes, com vagas 0 e com e-mail sem @, e tratada
04 O programa seguindo normalmente depois de cada erro

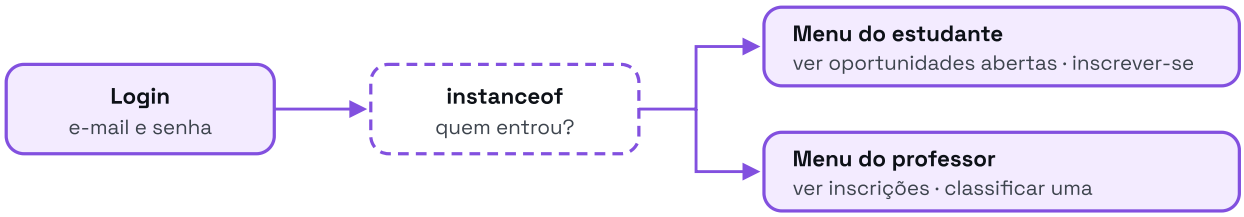
Final

Entrega final e defesa

ArrayList<T> · HashMap<K, V> · Scanner · instanceof

18/12
sexta · com a P²

Coleções com tipo e um menu curto fecham o sistema.



Exigências

- [ ] Um `HashMap<String, Usuario>` com os usuários pelo e-mail, usado no login
- [ ] Um `ArrayList<Oportunidade>` com todas as oportunidades
- [ ] Menu de console com `Scanner`: login por e-mail e senha, e `instanceof` decide se abre o menu de professor ou o de estudante
- [ ] Menu do estudante: ver as oportunidades abertas e se inscrever
- [ ] Menu do professor: ver as inscrições das suas oportunidades e classificar uma
- [ ] Dados iniciais criados no `Main`: 2 professores, 3 estudantes, uma monitoria e uma pesquisa
- [ ] Texto no lugar de número ou vaga esgotada não derruba o programa: o erro é tratado e o menu volta

A defesa

10 minutos por equipe, no laboratório

1. **A equipe demonstra o fluxo**
   o estudante se inscreve, o professor classifica, e a vaga esgotada aparece tratada
2. **Uma pergunta para cada integrante**
   sobre qualquer parte do código, escolhida pelo professor na hora
3. **Nota individual**
   cada um responde pela própria defesa

★ Ponto extra, um por item

- [ ] Uma classe genérica `Repositorio<T>` no lugar das listas soltas
- [ ] As oportunidades em ordem alfabética com `Comparable`
- [ ] Uma interface gráfica (Swing ou JavaFX), que não roda no OnlineGDB e precisa de uma IDE instalada